

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ YILMAZ
Nom d'usage ▶ YILMAZ
Prénom ▶ Ceyhane
Adresse ▶ 29 rue du Vulture, 12100 Millau

Titre professionnel visé

Concepteur Développeur d'Applications

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.
Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer une application sécurisée

p.

- **Projet Fil Rouge Jalon 1** : développement du site web dynamique. p. 5
- **Projet Fil Rouge Jalon2** : développement de l'application desktop WinForms. p. 7
- **Stage** : développement de l'Oracle Data API pour le Centre Hospitalier de Mende. p. 9

Concevoir et développer une application sécurisée organisée en couches

p.

- **Projet Fil Rouge Jalon 2** : conception de l'API REST sécurisée. p. 11
- **Stage** : conception du pipeline ETL en architecture multicouches. p. 13
- **Stage** : conception de la base de données relationnelle PostgreSQL pour les données hospitalières. p. 15

Préparer le déploiement d'une application sécurisée

p.

- **Projet Fil Rouge Jalon 2** : conteneurisation et CI/CD de la Cookbook API avec Docker et GitHub Actions. p. 17
- **Projet Fil Rouge Jalon 2** : plan de tests automatisés de l'API REST. p. 19
- **Stage** : plan de déploiement du pipeline de données hospitalières sur Azure. p. 21

Titres, diplômes, CQP, attestations de formation *(facultatif)*

p. 23

Déclaration sur l'honneur

p. 24

Documents illustrant la pratique professionnelle *(facultatif)*

p. 25

Annexes *(Si le RC le prévoit)*

p. 26

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer une application sécurisée

Exemple n°1 ► Développement du site web dynamique.

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du premier jalon du projet fil rouge de ma formation CDA, j'ai développé un site web dynamique et sécurisé pour une entreprise fictive nommée « Gastronomes Créatifs », une plateforme de partage de recettes de cuisine.

Les principales tâches réalisées sont les suivantes :

- **Développement de l'interface utilisateur (Front-end)** : minimum 6 pages HTML avec CSS « mobile first » pour garantir le responsive design, Inscription/Connexion, Accueil (avec un carrousel des meilleures recettes), Recherche, Détail d'une recette, Ajout/Modification de recette, et Administration.
- **Développement des composants métier (Back-end)** : j'ai développé les contrôleurs ASP.NET Core MVC en C# pour gérer la logique applicative : gestion des utilisateurs (inscription, connexion, accès conditionnel), CRUD des recettes (création, lecture, modification), système de notation par étoiles (1 à 5) et commentaires.
- **Recherche dynamique** : j'ai implémenté une fonctionnalité de recherche utilisant l'API Fetch en JavaScript, permettant aux utilisateurs de rechercher des recettes par mots-clés et de filtrer par critères avancés.
- **Gestion de l'authentification** : j'ai mis en place un système d'accès conditionnel : seule la page d'accueil est accessible sans connexion, les autres fonctionnalités nécessitent une inscription et une connexion.
- **Sécurisation de l'application** : j'ai protégé l'application contre les injections SQL en utilisant des requêtes paramétrées via l'ORM Dapper. J'ai également mis en place des tokens anti-CSRF.
- **Conformité RGPD** : j'ai veillé à ne pas afficher publiquement les données personnelles des utilisateurs et j'ai informé les utilisateurs de l'utilisation des cookies.
- **Gestion des erreurs** : j'ai prévu un traitement pour les pages non trouvées (erreur 404).

2. Précisez les moyens utilisés :

- **Langages** : C#, HTML5, CSS3, JavaScript
- **Framework Back-end** : ASP.NET Core MVC
- **ORM** : Dapper (pour les requêtes paramétrées et le mapping objet-relationnel)
- **SGBD** : PostgreSQL (conteneurisé avec Docker)
- **IDE** : Visual Studio (Back-end), Visual Studio Code (Front-end)
- **Gestion de version** : Git/GitHub
- **Prototypage** : Penpot
- **Gestion de projet** : GitHub Projects (tableau Kanban)
- **Communication** : Microsoft Teams
- **Validation** : outils du W3C pour la validation HTML/CSS

3. Avec qui avez-vous travaillé ?

J'ai travaillé en autonomie sur ce projet, sous la supervision de mes formateurs de l'organisme de formation 2iSA. Les formateurs jouaient le rôle de maîtrise d'ouvrage et étaient disponibles pour des clarifications et des revues de code. J'ai également échangé régulièrement avec mes collègues de promotion pour l'entraide et les échanges techniques.

4. Contexte

Nom de l'entreprise, organisme ou association ► 2iSA

Chantier, atelier, service ► Formation Concepteur développeur d'Applications (CDA)

Période d'exercice ► Du 20/01/2025 au 02/05/2025

5. Informations complémentaires (facultatif)

Ce projet m'a permis de consolider mes compétences en développement Front-end et Back-end. La principale difficulté rencontrée a été la gestion du temps et l'organisation initiale du projet, avec une tendance à la procrastination. J'ai partiellement corrigé ce point en mettant en place un tableau Kanban sur GitHub Projects pour structurer mon travail. Les fonctionnalités bonus implémentées incluent un carrousel de recettes populaires sur la page d'accueil.

Activité-type 1 Développer une application sécurisée

Exemple n°2 ► Développement de l'application desktop WinForms

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du second jalon du projet fil rouge, j'ai développé une application desktop WinForms en C# qui consomme l'API REST « Cookbook API » (également développée dans ce jalon). L'application permet la gestion et la consultation de recettes pour l'entreprise fictive « Gastronomes Créatifs ».

Les tâches réalisées :

- **Développement de l'interface desktop WinForms** : j'ai conçu une interface graphique comprenant plusieurs onglets : « Account » (connexion), « Recipes » (consultation), « Categories » (gestion) et « Association manager » (gestion des liens recettes/catégories).
- **Intégration avec l'API REST** : l'application se connecte à l'API REST hébergée dans un conteneur Docker. Elle envoie des requêtes HTTP et gère les réponses JSON pour afficher les données.
- **Authentification JWT** : au lancement, l'utilisateur renseigne l'URL de l'API, son nom d'utilisateur et son mot de passe. Après connexion réussie, l'API renvoie un token JWT que l'application stocke et utilise pour toutes les requêtes suivantes. Le token déverrouille l'accès aux onglets protégés.
- **Gestion des rôles** : l'application distingue les utilisateurs standards (consultation) des administrateurs (gestion des catégories et associations). Les rôles sont intégrés dans le token JWT.
- **Application des règles de gestion** : j'ai implémenté côté client la gestion des erreurs métier retournées par l'API, notamment le refus de suppression d'une catégorie si c'est la dernière associée à une recette.

2. Précisez les moyens utilisés :

- **Langage** : c#
- **Framework Front-end desktop** : WinForms (.NET)
- **Framework Back-end** : ASP.NET Core Web API (API REST consommée)
- **Authentification** : JSON Web Tokens (JWT)
- **IDE** : Visual Studio/JetBrains Rider
- **Conteneurisation** : Docker (API et PostgreSQL)
- **Gestion de version** : Git/GitHub

3. Avec qui avez-vous travaillé ?

J'ai travaillé en autonomie sous la supervision de mes formateurs de l'organisme de formation 2iSA, qui assuraient le rôle de maîtrise d'ouvrage.

4. Contexte

Nom de l'entreprise, organisme ou association ► 2iSA

Chantier, atelier, service ► Formation Concepteur développeur d'Applications (CDA)

Période d'exercice ► Du 05/06/2025 au 12/11/2025

5. Informations complémentaires *(facultatif)*

Ce projet m'a permis de découvrir le développement d'applications « client lourd » avec WinForms et de comprendre la communication entre un client desktop et une API REST. La gestion des tokens JWT côté client et la distinction des rôles ont été des apprentissages clés. J'ai dû me familiariser avec le Framework WinForms, mais l'architecture multicouches de l'API REST m'était déjà familière grâce au MVC du Jalon 1.

Activité-type 1 Développer une application sécurisée

Exemple n°3 ► Développement de l'Oracle Data API pour le Centre Hospitalier de Mende

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Lors de mon stage au Centre Hospitalier de Mende, j'ai développé une API REST complète avec FastAPI (Python) pour exposer les données de la base Oracle HM (Hopital Manager) du service des urgences. Cette API constitue la couche d'extraction du système de réplication de données Oracle → PostgreSQL.

Les tâches réalisées :

- **Développement de l'API REST avec FastAPI** : j'ai créé plusieurs endpoints : `/api/tables/{table_name}` (données), `/health`, `/metrics`. L'API supporte deux formats de réponse : JSON et MessagePack.
- **Protection contre l'injection SQL** : tous les identifiants SQL sont validés par une expression régulière avant d'être utilisés dans les requêtes. Les valeurs de paramètres sont passées via des bind parameters.
- **Gestion des secrets avec Azure Key Vault** : aucun secret n'est stocké dans le code source. Tous les identifiants de connexion sont récupérés dynamiquement depuis Azure Key Vault.
- **Pagination par curseur (keyset pagination)** : j'ai implémenté une pagination basée sur la combinaison plutôt que OFFSET/LIMIT, garantissant des performances constantes même sur des millions de lignes.
- **Gestion des types Oracle complexes** : j'ai développé un `output_type_handler` pour convertir les LOBs (BLOBs, CLOBs) directement en types natifs Python, évitant un appel réseau supplémentaire par objet.
- **Pool de connexions** : j'ai configuré un pool de connexions Oracle (min=2, max=10) pour optimiser les performances et éviter la création répétée de connexions coûteuses.
- **Rate limiting** : j'ai implémenté un rate limiting via la bibliothèque SlowAPI (60/min pour les endpoints de données) pour protéger l'API contre les abus.
- **Observabilité** : j'ai instrumenté l'API avec Prometheus pour le monitoring en production.

2. Précisez les moyens utilisés :

- **Langage** : python
- **Framework** : FastAPI
- **Base de données source** : Oracle Database (HM — Hopital Manager)
- **Driver Oracle** : oracledb (avec pool de connexions)
- **Gestion des secrets** : Azure Key Vault + DefaultAzureCredential
- **Rate limiting** : SlowAPI
- **Monitoring** : Prometheus (prometheus-fastapi-instrumentator)
- **Sérialisation** : JSON, MessagePack
- **Validation** : Pydantic, JSON Schema
- **IDE** : Visual Studio Code
- **Gestion de version** : Git/Azure Repos

3. Avec qui avez-vous travaillé ?

J'ai travaillé sous la supervision de Ma maîtresse de stage, ma maîtresse de stage et cheffe de projet Data Management au sein du pôle régulation et intelligence artificielle du Centre Hospitalier. Elle jouait le rôle de maîtrise d'ouvrage. Nos échanges se faisaient principalement via Microsoft Teams, avec des points réguliers et des récapitulatifs d'activité quotidiens. Le service informatique de l'hôpital et l'administrateur de la base Oracle HM ont été sollicités ponctuellement pour les accès réseau et les autorisations.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Hôpital Lozère

Chantier, atelier, service ▶ Direction des Systèmes Informatiques

Période d'exercice ▶ Du 24/11/2025 au 13/02/2026

5. Informations complémentaires (facultatif)

Ce projet m'a plongé dans le domaine du data engineering, un domaine entièrement nouveau pour moi. L'API Oracle est née d'une nécessité pratique : l'impossibilité de faire fonctionner le connecteur Oracle de Meltano (un outil de pipeline de données). Elle a rapidement démontré sa valeur intrinsèque en découplant complètement le pipeline de la base Oracle de production. La configuration des tables se fait de manière déclarative via des fichiers JSON, permettant l'ajout de nouvelles tables sans modification du code.

Activité-type 2

Concevoir et développer une application sécurisée organisée en couches

Exemple n°1 ► Conception de l'API REST sécurisée

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du Jalon 2 du projet fil rouge, j'ai conçu et développé une API REST sécurisée en ASP.NET Core selon une architecture en couches stricte. L'objectif était de fournir les endpoints CRUD pour les recettes, ingrédients et catégories de la plateforme « Gastronomes Créatifs ».

Les tâches réalisées :

- **Analyse des besoins et maquetage** : j'ai analysé le cahier des charges pour identifier les besoins. J'ai produit un diagramme de cas d'utilisation UML et des maquettes pour l'application WinForms.
- **Définition de l'architecture logicielle en couches** :
 - **Couche API** : réception des requêtes HTTP, validation des entrées, retour des réponses JSON. Gestion de l'authentification et de l'autorisation.
 - **Couche Métier** : logique applicative et règles de gestion. Par exemple, vérifier qu'une recette a toujours au moins une catégorie avant de permettre la suppression d'une association.
 - **Couche d'Accès aux Données** : traduction des appels de la couche métier en requêtes SQL vers PostgreSQL.
- **Développement de la sécurité JWT** : j'ai implémenté l'authentification par JSON Web Tokens. Les endpoints sont protégés par des attributs pour la consultation et pour les actions sensibles.
- **Conception de la base de données relationnelle** : j'ai conçu le modèle de données et utilisé Entity Framework Core pour la gestion de la base PostgreSQL conteneurisée avec Docker.
- **Modélisation UML** : j'ai produit un diagramme de classes pour l'architecture de l'API et un diagramme de séquence illustrant le flux de suppression d'un lien recette-catégorie.

2. Précisez les moyens utilisés :

- **Langage** : C#
- **Framework** : ASP.NET Core Web API
- **ORM** : Entity Framework Core (EF Core)
- **SGBD** : PostgreSQL (conteneurisé avec Docker)
- **Authentification** : JSON Web Tokens (JWT)
- **IDE** : Visual Studio/JetBrains Rider
- **Gestion de version** : Git/GitHub
- **Modélisation** : UML (diagrammes de cas d'utilisation, de classes, de séquence)

3. Avec qui avez-vous travaillé ?

J'ai travaillé en autonomie sous la supervision de mes formateurs de l'organisme 2iSA, qui assuraient le rôle de maîtrise d'ouvrage.

4. Contexte

Nom de l'entreprise, organisme ou association ► 2iSA

Chantier, atelier, service ► Formation Concepteur Développeur d'Applications (CDA)

Période d'exercice ► Du 05/06/2025 au 12/11/2025

5. Informations complémentaires (facultatif)

L'architecture en couches découplées est essentielle : l'application WinForms (le client) communique uniquement avec la couche API, qui elle-même ne connaît que la couche métier (BLL), et seule la BLL dialogue avec la DAL. Ce découplage garantit la maintenabilité, la testabilité et la réutilisabilité du code. Le diagramme de séquence de la suppression d'une association recette-catégorie illustre parfaitement ce principe : le contrôleur valide la requête, le service applique la règle de gestion (« une recette doit avoir au moins une catégorie »), et le contexte EF Core exécute l'opération en base.

Activité-type 2

Concevoir et développer une application sécurisée organisée en couches

Exemple n°2 ► Conception du pipeline ETL en architecture multicouches

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Lors de mon stage au Centre Hospitalier de Mende, j'ai conçu et développé un pipeline ETL en Python pour synchroniser les données de la base Oracle HM vers PostgreSQL Azure. Ce pipeline est construit selon une architecture en couches.

Les tâches réalisées :

- **Définition de l'architecture logicielle** : j'ai structuré le pipeline selon une architecture en couches avec injection de dépendances :
 - **Couche Orchestration** : coordonne le flux de données.
 - **Couche Extraction** : consomme l'Oracle Data API via des requêtes HTTP.
 - **Couche Transformation** : normalise les types de données Oracle vers des types compatibles.
 - **Couche Chargement** : plusieurs implémentations interchangeables via le pattern Strategy.
- **Injection de dépendances** : les composants du pipeline sont créés puis injectés dans l'orchestrateur, permettant de changer de stratégie de chargement sans modifier le code du pipeline.
- **Gestion des secrets** : utilisation d'un SecretsManager qui récupère les identifiants depuis Azure Key Vault.
- **Développement des composants d'accès aux données** : j'ai développé l'accès à deux SGBD : oracle en lecture seule et PostgreSQL Azure en lecture et écriture.
- **Vérification d'intégrité post-chargement** : le pipeline compare les données chargées avec les données source. En cas de divergence, le curseur n'est pas mis à jour, forçant un rechargement.
- **Métriques et monitoring** : chaque exécution produit un rapport de métriques (mémoire, nombre de lignes, timing par étape) pour détecter les fuites mémoire et les goulots d'étranglement.

2. Précisez les moyens utilisés :

- **Langage** : python
- **Bibliothèque de données** : Polars
- **Framework API** : FastAPI (pour l'API de synchronisation)
- **Base de données source** : Oracle Database
- **Base de données cible** : PostgreSQL Azure
- **ORM/Drivers** : SQLAlchemy, oracledb, psycopg
- **Gestion des secrets** : Azure Key Vault
- **Validation** : Pydantic, JSON Schema
- **Logging** : Loguru
- **IDE** : Visual Studio Code
- **Gestion de version** : Git/Azure Repos

3. Avec qui avez-vous travaillé ?

J'ai travaillé sous la supervision de Ma maîtresse de stage, cheffe de projet Data Management. Marine avait initié un travail de synchronisation Oracle → PostgreSQL avant mon arrivée. Suite à une comparaison de nos deux projets, elle a décidé que je devais intégrer mes améliorations techniques (architecture en couches, stratégies de chargement, COPY UPSERT, métriques, vérification d'intégrité) dans son projet existant, capitalisant sur les forces de chaque approche.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Hôpital Lozère

Chantier, atelier, service ▶ Direction des Systèmes Informatiques (DSI)

Période d'exercice ▶ Du 24/11/2025 au 13/02/2026

5. Informations complémentaires (facultatif)

Le choix de Polars plutôt que Pandas s'est avéré pertinent : certaines tables comptent plus de 5 millions de lignes. Polars, écrit en Rust, offre une exécution parallèle native, une évaluation paresseuse et une empreinte mémoire plus faible grâce au format Apache Arrow. J'ai également exploré deux autres approches : Meltano (un Framework de pipelines de données basé sur Singer, abandonné car trop complexe) et dlt (data load tool), découvert tardivement mais représentant le meilleur compromis entre flexibilité et simplicité.

Activité-type 2

Concevoir et développer une application sécurisée organisée en couches

Exemple n°3 ► Conception de la base de données relationnelle PostgreSQL pour les données hospitalières

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du même projet de stage, j'ai contribué à la conception et à la mise en place de la base de données relationnelle PostgreSQL Azure destinée à recevoir les données répliquées depuis Oracle HM.

Les tâches réalisées :

- **Analyse des besoins en données** : à partir de l'expression des besoins fonctionnelle rédigée par Ma maîtresse de stage, j'ai identifié les tâches à réaliser.
- **Configuration des tables** : j'ai contribué au renforcement de la validation des configurations des 84 tables via un modèle Pydantic (TableConfig). Ce modèle définit pour chaque table : le nom, la clé primaire, la colonne de date de modification, le schéma cible PostgreSQL, et les colonnes spécifiques à synchroniser. Un validateur garantit que la clé primaire figure toujours dans les colonnes synchronisées.
- **Développement de composants d'accès aux données SQL** : j'ai implémenté le protocole COPY natif de PostgreSQL pour le chargement en masse. Ce mécanisme crée une table temporaire, y charge les données, puis fusionne les données dans la table cible.
- **Minimisation des données (RGPD)** : le paramètre `sync_columns` dans la configuration permet d'exclure les colonnes non nécessaires. Par exemple, la colonne `DOCU_BLOB_CONTENU` (contenu binaire des documents) est explicitement exclue de la synchronisation.
- **Gestion de l'incrémentalité** : j'ai implémenté un mécanisme de curseur basé sur la date de dernière modification (`last_updated`), persisté dans un fichier JSON. À chaque exécution, seules les données modifiées depuis la dernière synchronisation sont extraites et chargées.

2. Précisez les moyens utilisés :

- **SGBD** : PostgreSQL et Oracle Database
- **Langage** : Python
- **ORM/Drivers** : SQLAlchemy, psycopg (protocole COPY), oracledb
- **Validation** : Pydantic
- **Bibliothèque de données** : Polars
- **Sécurité** : chiffrement SSL/TLS en transit, AES-256 au repos (Azure), Azure Active Directory pour le contrôle d'accès (en production)

3. Avec qui avez-vous travaillé ?

J'ai travaillé avec ma maîtresse de stage et ponctuellement avec le service informatique de l'hôpital et l'administrateur de la base Oracle HM pour les autorisations d'accès.

4. Contexte

Nom de l'entreprise, organisme ou association ► Hôpital Lozère

Chantier, atelier, service ► Direction des Systèmes Informatiques (DSI)

Période d'exercice ► Du 24/11/2025 au 13/02/2026

5. Informations complémentaires (facultatif)

La sensibilité des données hospitalières (identifiants patients pseudonymisés, diagnostics médicaux en codes CIM-10, dates de passages aux urgences) a rendu la conformité RGPD particulièrement critique. Les logs du pipeline ne contiennent jamais de données patients : seuls les noms de tables, les compteurs de lignes et les métriques de performance sont journalisés.

Activité-type 3

Préparer le déploiement d'une application sécurisée

Exemple n°1 ► Conteneurisation et CI/CD de la Cookbook API avec Docker et GitHub Actions

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du Jalon 2 du Fil Rouge, j'ai préparé et documenté le déploiement de l'API REST « Cookbook API » en mettant en place une conteneurisation Docker et un pipeline CI/CD complet avec GitHub Actions.

Les tâches réalisées :

- **Conteneurisation avec Docker Compose** : j'ai rédigé un fichier docker-compose.yml définissant les services :
 - Un conteneur pour la base de données PostgreSQL avec un volume Docker persistant.
 - Un conteneur pour l'API REST, construit à partir d'un Dockerfile, exposant le port 8080.
 - Un réseau Docker privé pour la communication sécurisée et isolée entre l'API et la base de données.
- **Pipeline d'Intégration Continue (CI)** : ce workflow GitHub Actions se déclenche à chaque push ou pull_request sur la branche master. Il installe .NET, restaure les dépendances, compile en mode Release, et exécute l'ensemble des tests. Si un seul test échoue, le pipeline bloque la fusion.
- **Pipeline de Déploiement Continu (CD)** : ce workflow se déclenche uniquement lors d'un push sur la branche release ou lors de la création d'un tag de version (ex : v1.0.0). Il se connecte à la registry GitHub Container Registry (ghcr.io) avec un GITHUB_TOKEN sécurisé, construit l'image Docker via docker/build-push-action, la tagge (ex : ghcr.io/ceynou/cookbook:latest), la pousse vers la registry, et la signe avec cosign pour garantir son authenticité et son intégrité.
- **Documentation du déploiement** : j'ai rédigé la documentation décrivant la procédure de mise en production, les prérequis et la configuration.

2. Précisez les moyens utilisés :

- **Conteneurisation** : Docker, Docker Compose
- **CI/CD** : GitHub Actions (workflows YAML)
- **Registry** : GitHub Container Registry (ghcr.io)
- **Signature d'image** : cosign
- **Framework** : ASP.NET Core Web API, .NET 10 Preview
- **Gestion de version** : Git/GitHub
- **SGBD** : PostgreSQL (Docker)

3. Avec qui avez-vous travaillé ?

J'ai travaillé en autonomie sous la supervision de mes formateurs de 2iSA.

4. Contexte

Nom de l'entreprise, organisme ou association ► 2iSA

Chantier, atelier, service ► Formation Concepteur Développeur d'Applications (CDA)

Période d'exercice ► Du 05/06/2025 au 12/11/2025

5. Informations complémentaires (facultatif)

Ce pipeline CI/CD automatise entièrement le processus, des tests à la publication, constituant le cœur de la compétence « Contribuer à la mise en production dans une démarche DevOps ». La conteneurisation est également une pratique fondamentale du Green IT : elle permet une mutualisation et une utilisation plus dense des ressources d'un serveur physique, réduisant le nombre de machines nécessaires.

Activité-type 3 Préparer le déploiement d'une application sécurisée

Exemple n°2 ► Plan de tests automatisés de l'API REST

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai préparé et exécuté un plan de tests complet pour l'API REST « Cookbook API » du Jalon 2, en mettant en place une stratégie de tests automatisés à plusieurs niveaux.

Les tâches réalisées :

- **Tests unitaires** : j'ai ciblé la logique métier pure en isolant les services de la base de données :
 - **PasswordHasherService** : hachage et vérification des mots de passe.
 - **CookbookService** : validation des règles de gestion.
 - **AuthenticationController** : logique d'authentification.
- **Tests d'intégration** : j'ai validé le comportement complet de l'application, de la requête HTTP jusqu'à la base de données, en utilisant Testcontainers pour créer à la volée une base PostgreSQL éphémère dans Docker :
 - **RecipesController** : un test complet qui crée une recette en tant qu'admin, la lit, la modifie et la supprime, en vérifiant les codes HTTP et les données à chaque étape.
 - **AccessService/AuthenticationController** : tests simulant une inscription suivie d'une connexion, et vérification de la gestion des doublons.
- **Tests de sécurité** : vérification que l'accès est refusé sans authentification ou avec un rôle insuffisant.
- **Exécution automatisée** : les tests sont exécutés automatiquement à chaque push et pull_request sur GitHub grâce au workflow GitHub Actions (dotnet.yml).

2. Précisez les moyens utilisés :

- **Framework de test** : xUnit
- **Conteneurs de test** : Testcontainers (base PostgreSQL éphémère dans Docker)
- **Mocking** : Mocks pour isoler la logique métier de la base de données
- **CI/CD** : GitHub Actions (exécution automatique des tests)
- **Langage** : C#
- **Framework** : ASP.NET Core Web API

3. Avec qui avez-vous travaillé ?

J'ai travaillé en autonomie sous la supervision de mes formateurs de 2iSA.

4. Contexte

Nom de l'entreprise, organisme ou association ► 2iSA

Chantier, atelier, service ► Formation Concepteur développeur d'Applications (CDA)

Période d'exercice ► Du 05/06/2025 au 12/11/2025

5. Informations complémentaires *(facultatif)*

L'utilisation de Testcontainers est particulièrement intéressante : elle crée à la volée une base de données PostgreSQL réelle dans un conteneur Docker pour les tests d'intégration, assurant un environnement de test propre et isolé. Cela évite les problèmes de dépendance à une base de données partagée et garantit la reproductibilité des tests. Ma stratégie de test couvre les trois aspects essentiels : la logique métier (unitaire), le flux complet (intégration) et la sécurité (autorisation/authentification).

Activité-type 3

Préparer le déploiement d'une application sécurisée

Exemple n°3 ► Plan de déploiement du pipeline de données hospitalières sur Azure

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Lors de mon stage, j'ai préparé le déploiement de l'ensemble du système de réplication de données et rédigé les plans de tests associés.

Les tâches réalisées :

- **Plan de tests du pipeline** : j'ai adopté une approche de test à trois niveaux :
 - **Unitaires** : validation des identifiants SQL, sérialisation des types Oracle, parsing des dates.
 - **Intégration** : appels API avec base de données mockée, pipeline complet avec données de test.
 - **Bout en bout** : flux complet de données de l'API Oracle jusqu'à PostgreSQL.
- **Préparation du déploiement de l'API Oracle** :
 - Création de l'environnement virtuel Python et installation des dépendances.
 - Configuration Azure (authentification via az login, vérification de l'accès au Key Vault).
 - Documentation du lancement en mode développement et en mode production.
 - Procédure de vérification post-déploiement (test de santé, liste des tables configurées).
- **Pipeline Azure DevOps (CI/CD)** : j'ai préparé un fichier azure-pipelines.yml pour l'intégration continue, comprenant l'installation de Python, l'installation des dépendances, l'exécution des tests avec couverture de code, et la publication des résultats.
- **Contribution à la démarche DevOps** : le pipeline CI/CD exécute les tests à chaque push, interceptant les bugs tôt et évitant des cycles de déploiement/redéploiement correctifs coûteux en ressources.
- **Veille technologique** : j'ai mené une veille sur la stack data moderne (dlt, SQLMesh, ClickHouse, Dagster) et documenté mes recommandations pour l'équipe du Centre Hospitalier.

2. Précisez les moyens utilisés :

- **Framework de test** : pytest, pytest-cov
- **CI/CD** : Azure DevOps Pipelines
- **Serveur ASGI** : uvicorn (mode développement et production)
- **Infrastructure** : Azure (Key Vault, PostgreSQL Azure)
- **Langage** : python
- **Framework** : FastAPI
- **Conteneurisation** : envisagé (Docker sur VM Azure)

3. Avec qui avez-vous travaillé ?

J'ai travaillé avec ma maîtresse de stage, les formateurs de 2iSA, le service informatique du Centre Hospitalier et l'équipe Azure.

4. Contexte

Nom de l'entreprise, organisme ou association ► Hôpital Lozère

Chantier, atelier, service ► Direction des Systèmes Informatiques (DSI)

Période d'exercice ► Du 24/11/2025 au 13/02/2026

5. Informations complémentaires (facultatif)

Le plan de déploiement est resté théorique par manque de temps pour le tester en conditions réelles. J'ai néanmoins documenté les principes d'éco-conception (Green IT) appliqués au projet : transfert optimisé via MessagePack (~25% plus compact que JSON), incrémentalité (ne transférer que les données modifiées), protocole COPY natif de PostgreSQL (réduit les allers-retours réseau), pool de connexions (réduit la charge serveur), et Polars plutôt que Pandas (consomme significativement moins de mémoire). J'ai également identifié des perspectives d'amélioration : adopter dlt comme outil principal d'ingestion, explorer ClickHouse comme base OLAP, et mettre en place Dagster pour l'orchestration.

Déclaration sur l'honneur

Je soussigné(e)*Ceyhane YILMAZ*..... ,

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à ..*Millau*..... le ..*17/02/2026*.....

pour faire valoir ce que de droit.

Signature :